

# **Hospital Appointment Management System**

Project Report

**Course:** Programming in Java

**Student Name:** Ram Raghuwanshi

**Registration No.:** 25BAI10166

**Year / Semester:** 2nd Year, Semester 1

**University:** VIT University

**Submitted For:** Vityarthi Flipped Course Project

## **1. Introduction**

This project is a console based Hospital Appointment Management System developed in Java language as an assignment for the Vityarthi Programming in Java course that utilizes Java core concepts including the Object-oriented programming, file handling, exceptions handling and Java Collections Framework in the practical use case. The system provides the clinic staff with the ability to register the patients, manage the doctor schedules, make and cancel appointments, as well as generate out consultation bills through the menu interface. The information is saved in plain text files, thus ensuring its persistence.

## **2. Problem Statement**

Small clinics often manage appointments and records manually using registers or spreadsheets. This leads to common problems like:

- Double booking of doctor time slots
- No organised way to store patient medical history
- No billing system at the end of a consultation
- Data being lost when the day's register is misplaced

This system solves these problems by providing a simple software solution that manages patients, doctors, and appointments digitally, with data saved to files after every change.

## **3. Functional Requirements**

The project has three functional modules:

### **Module 1: Patient Registration & Records**

- Register a new patient with name, age, gender, contact number, and medical history
- View the full list of patients or search by patient name or ID
- Update an existing patient's details

### **Module 2: Doctor & Schedule Management**

- Add a doctor with their specialization, consultation fee, and available time slots
- View all doctors and their currently available slots
- Time slots are automatically removed when booked and restored when an appointment is cancelled

### **Module 3: Appointment Booking & Billing**

- Book an appointment by selecting a patient, a doctor, and a time slot
- Cancel a scheduled appointment (the slot is given back to the doctor)

- View all appointments or filter by patient ID or doctor ID
- Mark an appointment as complete and generate a consultation bill including 18% GST

## **4. Non-functional Requirements**

### **Performance**

Data is loaded from files at startup and kept in memory. All operations like search, book, and view are instant with no noticeable delay.

### **Reliability**

Every change is written to the data files immediately. If the program closes unexpectedly, no data is lost.

### **Usability**

The interface is a simple numbered menu. The user only needs to type a number. Invalid input shows a clear error message and re-prompts instead of crashing.

### **Maintainability**

Code is split into packages: model, service, storage, and exceptions. Each class has a single responsibility. Adding a new feature does not require touching unrelated classes.

### **Error Handling**

Three custom checked exception classes handle specific error cases:

PatientNotFoundException, SlotUnavailableException, and

InvalidAppointmentException. These give the user meaningful messages instead of generic Java stack traces.

## **5. System Architecture**

The project uses a simple layered architecture:

- Presentation Layer: Main.java handles the console menu and all user input/output
- Business Logic Layer: HospitalService.java contains all operations and validation rules
- Storage Layer: FileHandler.java reads from and writes to the data text files
- Model Layer: Person, Patient, Doctor, Appointment classes represent the data objects

The flow is always top-down: the user interacts with Main, which calls HospitalService for logic, which calls FileHandler when data needs to be saved or loaded.

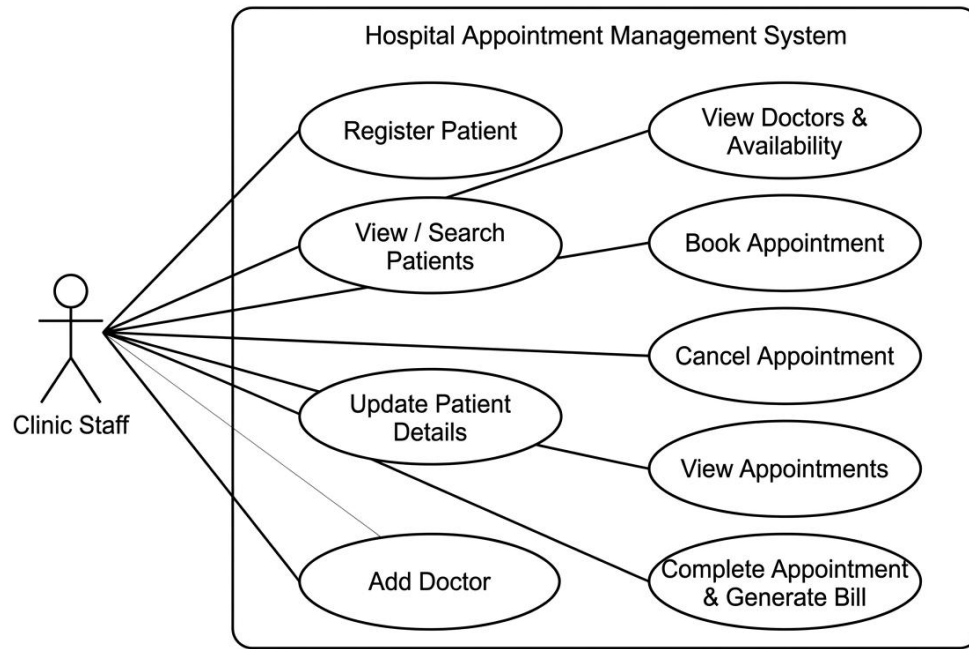
## **Package / Folder Structure**

```
src/  
  Main.java  
  model/  
    Person.java  
    Patient.java  
    Doctor.java  
    Appointment.java  
    Billable.java  
  service/  
    HospitalService.java  
  storage/  
    FileHandler.java  
  exceptions/  
    PatientNotFoundException.java  
    SlotUnavailableException.java  
    InvalidAppointmentException.java  
data/  
  patients.txt  
  doctors.txt  
  appointments.txt
```

## 6. Design Diagrams

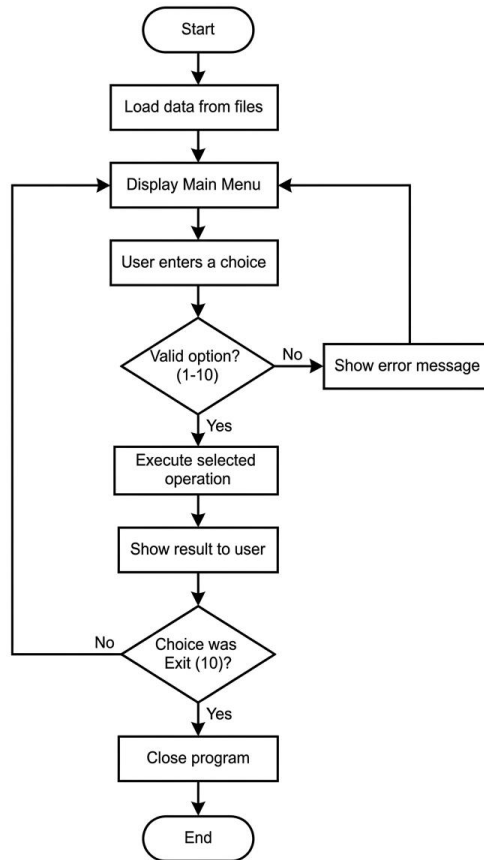
### 6.1 Use Case Diagram

#### Hospital Appointment Management System



The diagram shows one actor (Clinic Staff) interacting with nine use cases inside the system boundary. All operations are accessible from a single console menu.

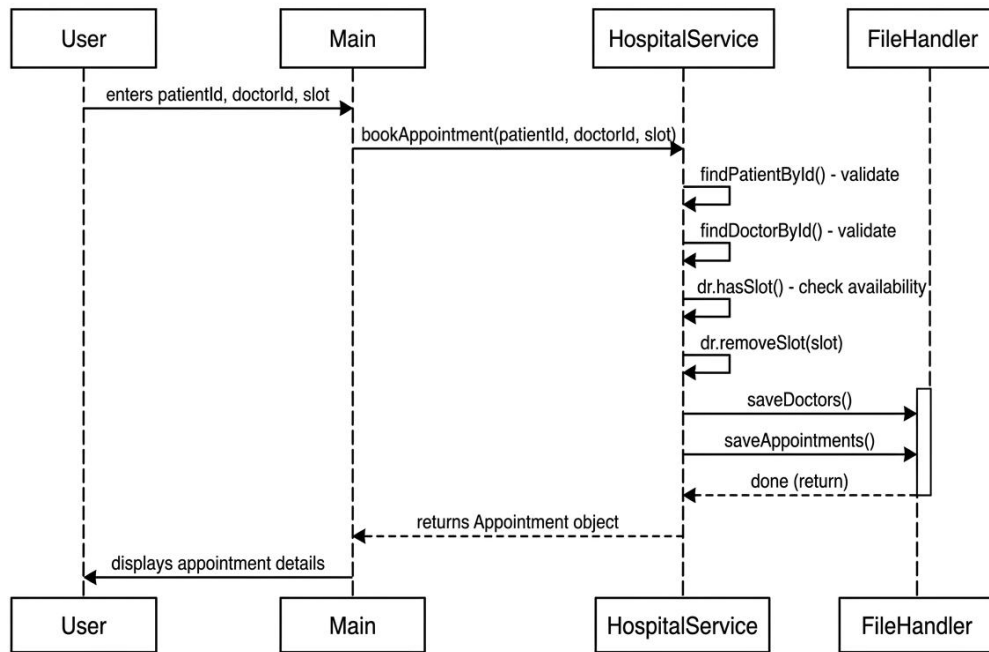
## 6.2 Workflow Diagram



The workflow shows how the program loops through the menu until the user selects Exit. Invalid choices show an error and loop back. Every valid choice runs the operation and shows the result before returning to the menu.

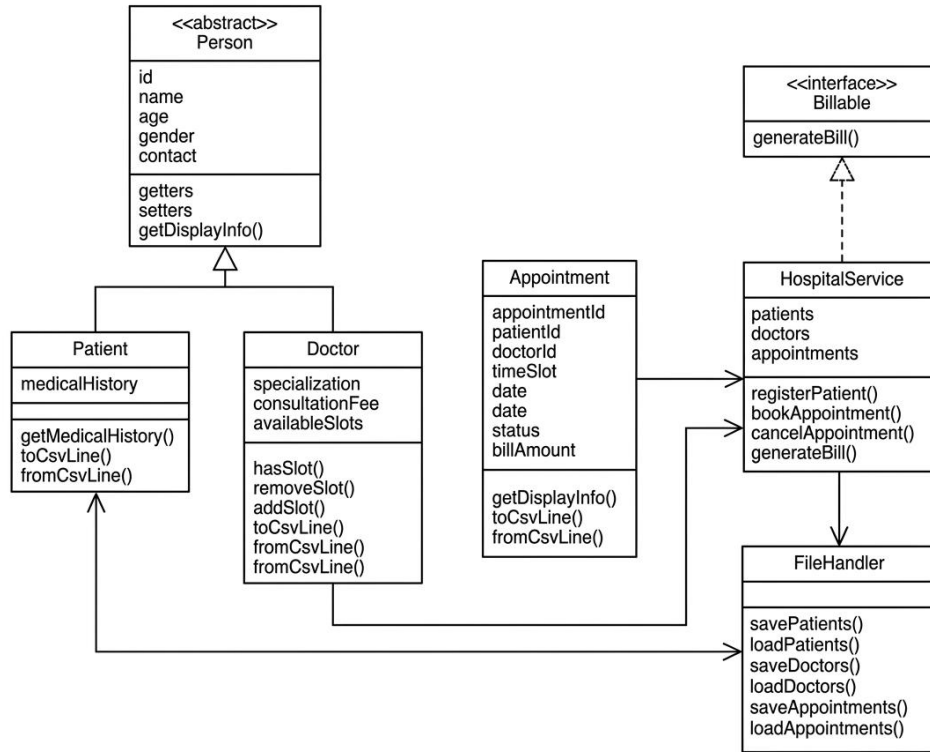
### 6.3 Sequence Diagram — Book Appointment

#### Sequence Diagram - Book Appointment



Shows the step-by-step message flow when a user books an appointment: from entering the IDs and slot, through validation in HospitalService, to saving data in FileHandler, and finally displaying the confirmed appointment.

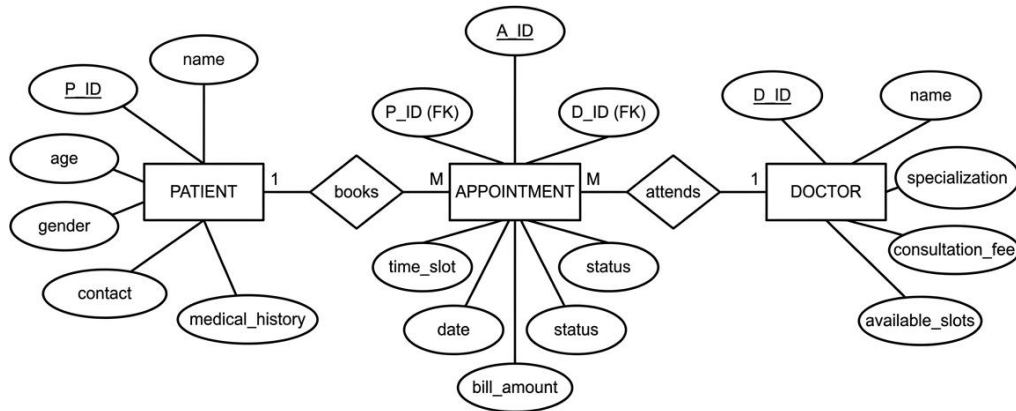
## 6.4 Class Diagram



Shows the full class structure of the project. Person is the abstract base class. Patient and Doctor inherit from it. HospitalService implements the Billable interface. FileHandler is used by HospitalService for all data persistence.

## 6.5 ER Diagram — Storage Design

ER Diagram - Data Storage Design



Shows the three data entities and their relationships. One Patient can have many Appointments. One Doctor can have many Appointments. The Appointment entity stores the foreign keys P\_ID and D\_ID to link them.

## 7. Design Decisions & Rationale

### Console-based interface

Chosen to keep focus on core Java like OOP, collections, exception handling rather than spending time on a GUI framework. It also keeps the code simple to read and evaluate.

### Pipe-delimited text files for storage

No external dependencies are needed. The pipe character (|) was chosen as delimiter because it does not appear in names or medical notes, unlike commas. Each object class handles its own serialization and deserialization through toCsvLine() and fromCsvLine() methods.

### **Abstract class Person instead of interface**

Person holds actual shared data fields (name, age, contact) and a constructor. An interface cannot hold instance state, so an abstract class was the correct choice to avoid repeating the same fields in both Patient and Doctor.

### **Custom checked exceptions**

Specific custom exceptions (PatientNotFoundException, SlotUnavailableException, InvalidAppointmentException) give meaningful error messages and force the calling code to handle each error case explicitly.

### **Billable as an interface**

Demonstrates the use of interfaces as required by the course. It also decouples bill generation from the main service logic, a different class could implement Billable for a different billing format without changing HospitalService.

## **8. Implementation Details**

### **OOP Concepts Applied**

- Inheritance: Patient and Doctor extend the abstract class Person
- Encapsulation: all fields are private, accessed only through getters and setters
- Abstraction: Person is an abstract class; Billable is an interface
- Polymorphism: getDisplayInfo() is overridden differently in Patient and Doctor

### **Java Features Used**

- ArrayList and List: for storing patients, doctors, and appointments in memory
- Streams and Lambdas: for searching/filtering collections cleanly
- Optional<T>: used in findDoctorById() to safely handle missing values without null checks
- LocalDate: used to automatically record the current date when an appointment is booked
- BufferedReader / BufferedWriter: for efficient file reading and writing
- String.format(): for formatted display of patient, doctor, and bill information
- Switch expressions (-> syntax): used in the main menu for cleaner code

### **Data Storage Format**

Each object is stored as one line in a text file using | as the field separator. For example:

```
P001|Amit Khanna|29|M|9876001234|Hypertension, on  
medication
```

D001|Priya

Sharma|42|F|9876543210|Cardiologist|800.0|09:00,10:00,11:00

## 9. Screenshots / Results

```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Ram_Raghuwanshi\Exams\VITEEE\vit\2ndYear\Sem 1\programming_in_Java\vityarthi>java -cp out Main

HOSPITAL APPOINTMENT MANAGEMENT SYSTEM

MAIN MENU
1. Register Patient
2. View / Search Patients
3. Update Patient Details
4. Add Doctor
5. View Doctors & Availability
6. Book Appointment
7. Cancel Appointment
8. View Appointments
9. Complete Appointment & Generate Bill
10. Exit

Enter choice: 2

VIEW / SEARCH PATIENTS
Search (name/ID) or press Enter to list all: P005

Found 1 patient(s):

ID      : P005
Name    : Atharv Kumar
Age     : 12
Gender  : M
Contact : 8733455511
History : Mild Hypertension

MAIN MENU
1. Register Patient
2. View / Search Patients
3. Update Patient Details
4. Add Doctor
5. View Doctors & Availability
```

6. Book Appointment
7. Cancel Appointment
8. View Appointments
9. Complete Appointment & Generate Bill
10. Exit

Enter choice: 6

#### BOOK APPOINTMENT

##### DOCTORS & AVAILABILITY

ID : D001  
Name : Priya Sharma  
Specialization : Cardiologist  
Consultation Fee: Rs. 800.00  
Available Slots: [09:00, 10:00, 11:00, 14:00, 15:00]

ID : D002  
Name : Rohan Mehta  
Specialization : General Physician  
Consultation Fee: Rs. 400.00  
Available Slots: [08:00, 09:00, 10:00, 11:00, 16:00]

ID : D003  
Name : Anjali Verma  
Specialization : Orthopedic Surgeon  
Consultation Fee: Rs. 1200.00  
Available Slots: [11:00, 14:00, 15:00, 16:00]

ID : D004  
Name : Vikram Nair  
Specialization : Dermatologist  
Consultation Fee: Rs. 600.00  
Available Slots: [09:00, 13:00, 14:00, 17:00]

ID : D005  
Name : Sunita Rao  
Specialization : Pediatrician  
Consultation Fee: Rs. 500.00  
Available Slots: [08:00, 09:00, 10:00, 11:00, 15:00, 16:00]

Patient ID (e.g. P001): P005  
Doctor ID (e.g. D001): D005  
Time Slot (e.g. 09:00): 15:00

[OK] Appointment booked!  
Appt ID : A005  
Patient ID: P005  
Doctor ID : D005  
Date/Slot : 2026-09-18 at 15:00  
Status : SCHEDULED  
Bill : Rs. 0.00

#### MAIN MENU

1. Register Patient
2. View / Search Patients
3. Update Patient Details
4. Add Doctor
5. View Doctors & Availability
6. Book Appointment
7. Cancel Appointment
8. View Appointments
9. Complete Appointment & Generate Bill
10. Exit

Enter choice: 9

#### COMPLETE APPOINTMENT & GENERATE BILL

Appointment ID (e.g. A001): A005

#### CONSULTATION BILL

Appointment ID : A005  
Date / Slot : 2026-09-18 at 15:00

Patient : Atharv Kumar  
Patient ID : P005

Doctor : Dr. Sunita Rao  
Specialization : Pediatrician

```
Consultation Fee : Rs. 500.00
GST (18%)       : Rs. 90.00
Total Payable   : Rs. 590.00
```

Thank you for visiting City Clinic!

#### MAIN MENU

1. Register Patient
2. View / Search Patients
3. Update Patient Details
4. Add Doctor
5. View Doctors & Availability
6. Book Appointment
7. Cancel Appointment
8. View Appointments
9. Complete Appointment & Generate Bill
10. Exit

Enter choice: 10

Goodbye! Stay Healthy.

C:\Ram\_Raghuwanshi\Exams\VITEEE\vit\2ndYear\Sem 1\programming\_in\_Java\vityarthi>

## 10. Testing Approach

The application was tested manually by running each menu option and verifying the output on screen and in the data files. Test cases covered:

**Register patient with valid data:** Patient created, assigned ID P001, saved to patients.txt

**Register patient with blank name:** Error message shown, no record created

**Search patient by partial name:** Matching patients listed correctly

**Book appointment with valid slot:** Appointment created, slot removed from doctor

**Book appointment with already-booked slot:** Error: slot not available shown

**Cancel a scheduled appointment:** Status set to CANCELLED, slot restored to doctor

**Try to cancel a completed appointment:** Error: cannot cancel shown

**Complete appointment:** Status set to COMPLETED, bill printed with GST calculation

**Filter appointments by patient ID:** Only that patient's appointments shown

**Restart program and check data:** All records reloaded correctly from files

## 11. Challenges Faced

**Double booking issue:** Initially there was no check to prevent the same slot from being booked twice. Fixed by removing the slot from the doctor's list immediately when a booking is made.

**ID counter reset on reload:** When the program restarted, the counter was starting from 1 again, which would generate duplicate IDs. Fixed by seeding the counter from the maximum ID found in the loaded data.

**File path not found:** The program could not find the data/ folder when run from the wrong directory. Fixed by always running from the project root using `java -cp out Main`.

**Comma in medical history breaking the CSV:** Patient notes like "Hypertension, on medication" broke comma-delimited parsing. Switched to pipe (|) as the delimiter to solve this.

**Slot not restored on cancel:** When an appointment was cancelled, the doctor's slot was not being added back. Fixed by calling `doctor.addSlot()` inside `cancelAppointment()` and persisting the change.

## 12. Learnings & Key Takeaways

- Inheritance and abstraction make much more sense when you actually use them in code. The `Person` -> `Patient/Doctor` structure felt natural once implemented.
- Java Streams and Lambdas are much cleaner than traditional for loops for searching and filtering.
- Custom exceptions make error handling specific and readable. The calling code always knows exactly what went wrong.
- File I/O in Java is straightforward once you understand `BufferedReader` and `BufferedWriter`. Designing a clean serialization format matters more than it seems.
- Splitting code into layers (presentation, service, storage, model) makes it much easier to find bugs and make changes without breaking other parts.
- The `Billable` interface showed me why interfaces are useful, they let you define a contract without caring about which class implements it.

### **13. Future Enhancements**

- Add a GUI using JavaFX to make the system accessible to non-technical staff
- Replace text file storage with a database like SQLite using JDBC
- Add user authentication with role-based access — Admin, Doctor, Receptionist
- Add appointment reminders via email or SMS integration
- Generate monthly reports — total appointments, revenue, most visited doctor
- Support multiple appointments per time slot by tracking room availability separately

### **14. References**

- [1] VITyarthi Course Material Programming in Java
- [2] Java File I/O Tutorial <https://docs.oracle.com/javase/tutorial/essential/io/>
- [3] Java Collections Framework <https://docs.oracle.com/javase/tutorial/collections/>
- [4] Java Exception Handling <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- [5] Effective Java by Joshua Bloch Addison-Wesley, 3rd Edition
- [6] Oracle Java SE Documentation <https://docs.oracle.com/en/java/>